

APLICAÇÃO DE MÉTODOS NUMÉRICOS PARA DESCOBERTA DE RAÍZES DE FUNÇÕES

ANDRÉ LUCAS GONÇALVES GOMES, ANDRIEL VINICIUS DE MEDEIROS
FERNANDES, MARÍA PAZ MARCATO

1. INTRODUÇÃO

A raiz de uma função f qualquer é um número real γ tal que $f(\gamma) = 0$. A depender do grau ou da complexidade do polinômio, obter as raízes desta função é um trabalho difícil e, por vezes, impossível, pois não existe um método direto que forneça tal número. Devido a casos assim, são adotados métodos numéricos que não vão garantir a solução exata, mas uma aproximação dentro de uma precisão dada, que, por muitas vezes, é suficiente para lidar com os problemas práticos, além de permitirem o ajuste da precisão para valores mais ou menos precisos de acordo com a natureza do problema.

Os métodos numéricos utilizados para resolver problemas de raízes de funções comumente são compostos por duas etapas:

- Etapa 1: Localizar todos os menores intervalos que contém raízes;
- Etapa 2: Dado um ponto inicial neste intervalo, refina-lo até se aproximar o suficiente da raiz real da função.

Para a etapa 1, é utilizado o Teorema de Bolzano descrito abaixo.

Theorem 1.1. *Se uma função contínua $f(x)$ assume valores de sinais opostos nos pontos extremos do intervalo $[a, b]$, isto é, $f(a) \cdot f(b) < 0$, então o intervalo conterá, no mínimo, uma raiz da equação.*

Como consequência desse teorema, temos o seguinte corolário: *Se a derivada de $f(x)$ preservar o sinal em $[a, b]$, então a raiz será única no intervalo.*

É importante notar que antes de utilizar qualquer método numérico é interessante que se tenha um conhecimento prévio do comportamento da função. Tal recomendação se justifica quando há métodos que podem não convergir em direção à raiz da função, ou seja, dado um γ , não há $f(\gamma) \approx 0$. Tal comportamento ocorre geralmente com um chute inicial ruim de um ponto.

2. IMPLEMENTAÇÃO

Para atingir o objetivo deste trabalho foram desenvolvidos algoritmos e scripts na linguagem Go. Cada problema descrito no trabalho foi armazenado em uma estrutura nomeada `Problem` que contém a função original, os pontos iniciais e finais do intervalo, um ID numérico associado ao problema e, por fim, a distância entre cada ponto do intervalo.

Para o refinamento das raízes, foram elaborados os métodos da Bisseção, Falsa Posição, Ponto Fixo, Newton-Raphson e Secante em um script desenvolvido com a linguagem Go.

2.1. Isolamento de Raízes.

Para a fase do Isolamento de raízes foi proposto um algoritmo autoral e simples, que deve receber uma instância de `Problem` e retornar uma lista de subintervalos, cada um contendo uma única raiz. O algoritmo faz isso aplicando o teorema de Bolzano, verificando se há mudança de sinal num intervalo contínuo da função e, logo após, checando a unicidade da raiz através do corolário: caso as derivadas numéricas dos pontos extremos do intervalo preservem o sinal, então a raiz é única; caso contrário, o intervalo pode conter mais de uma raiz e é reisolado de forma recursiva.

O método público `IsolateRoots` apenas delega o trabalho à função recursiva, repassando o passo de tabulação armazenado no campo `H` da instância.

```
func (self Problem) IsolateRoots() RootIntervallList {
    return isolateRec(self.Start, self.End, self.Function, self.H)
}
```

A função `isolateRec` percorre o intervalo pedindo a `nextInterval` o próximo par de pontos consecutivos do tabelamento entre os quais a função troca de sinal. Sobre esse par aplica-se primeiro o teorema de Bolzano e, em seguida, o corolário da unicidade, comparando o sinal das derivadas numéricas nos dois extremos.

```
func isolateRec(a, b float64, f Function, step float64) RootIntervallList {
    var collection = RootIntervalList{}
    var curr float64 = a

    for curr < b {
        sequence := nextInterval(curr, b, f, step)
        curr = sequence[len(sequence)-1]
        n := len(sequence)
        if n == 1 {
            continue
        }
        xmin := sequence[n-2]
        xmax := sequence[n-1]
        if f(xmin)*f(xmax) < 0 {
            d1 := deriv(xmin, f)
            d2 := deriv(xmax, f)
            if d1*d2 >= 0 {
                collection = append(collection, Interval{xmin, xmax})
            } else {
                subintervals := isolateRec(xmin, xmax, f, step)
                collection = append(collection, subintervals...)
            }
        }
    }

    return collection
}
```

O tabelamento propriamente dito fica em `nextInterval`, que caminha a partir de `a` em passos de tamanho `step` até encontrar uma troca de sinal em relação a $f(a)$ ou até alcançar `b`. O último par de pontos da sequência devolvida é o subintervalo candidato consumido por `isolateRec`.

```
func nextInterval(a, b float64, f Function, step float64) Interval {
    var slice Interval
    curr := a
    for curr < b && f(a)*f(curr) >= 0 {
        slice = append(slice, curr)
        curr += step
    }

    if curr <= b {
        slice = append(slice, curr)
    } else {
        slice = append(slice, b)
    }

    return slice
}
```

Cabe registrar uma limitação da estratégia recursiva adotada. Como a chamada recursiva reaproveita o mesmo passo `step`, ela não subdivide efetivamente um subintervalo que já tenha a largura de um passo: nesse caso `nextInterval` devolve o mesmo par de pontos, o corolário falha novamente e a recursão se repete sem progresso. Nos quatro problemas analisados a situação não se manifesta, pois o corolário é satisfeito em todos os subintervalos produzidos pelo tabelamento, mas uma versão mais robusta do algoritmo precisaria reduzir o passo a cada nível de recursão, por exemplo pela metade, de modo a garantir a terminação. Os intervalos efetivamente isolados em cada problema aparecem na Tabela 1 e estão representados graficamente na Figura 2.

2.2. Métodos Vistos em Aula.

Para as implementações descritas abaixo, foram elaboradas em geral 2 funções para cada algoritmo: uma função principal que carrega o nome do respectivo algoritmo e é pública para ser acessada, e outra auxiliar, de sufixo `candidate`, que contém o método de geração de raiz daquele algoritmo. O Ponto Fixo é a única exceção, por depender de uma terceira função, `pf_lambda`, encarregada de estimar a constante que define sua função de iteração. Todas as funções principais recebem uma instância de `Problem` e o valor da precisão, e retornam o valor da raiz encontrada e a quantidade de iterações; os métodos de Newton-Raphson, Secante e Ponto Fixo recebem ainda um limite máximo de iterações, por não operarem sobre um intervalo que encolhe a cada passo e, portanto, não dispõem de garantia natural de parada.

Alguns métodos requerem o uso de derivadas para seus cálculos. Para isso, fazemos o cálculo da derivada de maneira numérica, aplicando um h suficientemente próximo de 0 na fórmula $f'(x) = \frac{f(x+h)-f(x)}{h}$.

2.2.1. Bisseção.

A função principal `Bisection` é responsável pela atualização e checagem dos valores de acordo com a precisão e o critério do método da bisseção, que corta o intervalo ao meio a cada iteração. A escolha da raiz inicial é feita com a média dos valores `a` e `b` iniciais.

```
func Bisection(inst types.Problem, epsilon float64) (float64, uint16) {
    a_ := inst.Start
    b_ := inst.End
    var root float64
    var counter uint16 = 1

    for root = b_candidate(a_, b_); math.Abs(inst.Function(root)) > epsilon
    && (b_-a_) > epsilon; root = b_candidate(a_, b_) {
        if inst.Function(a_)*inst.Function(root) < 0 {
            b_ = root
        } else if inst.Function(b_)*inst.Function(root) < 0 {
            a_ = root
        }
        counter++
    }

    return root, counter
}
```

A função de iteração `b_candidate` recebe os valores de `a` e `b` atuais e retorna a média dos dois valores.

```
func b_candidate(a, b float64) float64 {
    return (a + b) / 2
}
```

2.2.2. *Falsa Posição.*

A função principal `FalsePosition` é responsável pela checagem e atualização dos valores de `a` e `b`, levando em consideração a precisão e os critérios do método da falsa posição, que divide o intervalo em dois a partir de uma média ponderada dos valores `a` e `b`, baseados em $f(a)$ e $f(b)$. O valor inicial escolhido para a raiz é estabelecido utilizando a própria função de iteração $\frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$.

```
func FalsePosition(inst types.Problem, epsilon float64) (float64, uint16)
{
    a_ := inst.Start
    b_ := inst.End
    fa_ := inst.Function(a_)
    fb_ := inst.Function(b_)
    root := fp_candidate(a_, b_, fa_, fb_)
    froot := inst.Function(root)
    var counter uint16 = 1

    for math.Abs(fb_-fa_) > epsilon && math.Abs(froot) > epsilon {
        if fa_*froot < 0 {
            b_ = root
            fb_ = froot
        } else {
            a_ = root
            fa_ = froot
        }
    }

    root = fp_candidate(a_, b_, fa_, fb_)
}
```

```

    froot = inst.Function(root)

    counter++
}
return root, counter
}

```

A função `fp_candidate` recebe os valores a , b , $f(a)$ e $f(b)$ e retorna o valor do próximo candidato a ser raiz baseado na função de iteração do método da falsa posição $\frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$.

```

func fp_candidate(a, b, fa, fb float64) float64 {
    return (a*fb - b*fa) / (fb - fa)
}

```

2.2.3. Ponto Fixo.

A função `FixedPoint` aplica o método do Ponto Fixo. Em vez de exigir que a função de iteração g seja fornecida manualmente, a implementação a constrói como $g(x) = x - \frac{f(x)}{\lambda}$, com λ estimado pela função `pf_lambda`. A aproximação inicial é o ponto médio de $[a, b]$, obtido por `b_candidate`.

```

func FixedPoint(inst types.Problem, epsilon float64, kmax uint16)
(float64, uint16) {
    lambda := pf_lambda(inst)
    root := b_candidate(inst.Start, inst.End)
    var counter uint16 = 1

    for math.Abs(inst.Function(root)) > epsilon && counter < kmax {
        root = pf_candidate(root, inst, lambda)
        counter++
    }

    return root, counter
}

```

A função `pf_candidate` calcula $x_{k+1} = x_k - \frac{f(x_k)}{\lambda}$.

```

func pf_candidate(x float64, inst types.Problem, lambda float64) float64 {
    return x - inst.Function(x)/lambda
}

```

Construção de g . Para qualquer constante $c \neq 0$,

$$f(x) = 0 \iff x + cf(x) = x,$$

logo $g(x) = x + cf(x)$ tem como pontos fixos as raízes de f . É o caso $A(x) = c$ da forma $g(x) = x + A(x)f(x)$.

Condição de convergência. Como $g'(x) = 1 + cf'(x)$, a condição $|g'(x)| < 1$ equivale a $-2 < cf'(x) < 0$. Ela limita c pelo inverso de f' , pois exige $|c| < \frac{2}{|f'(x)|}$. Adotando $c = -\frac{1}{\lambda}$, obtém-se

$$g(x) = x - \frac{f(x)}{\lambda}, \quad 0 < \frac{f'(x)}{\lambda} < 2.$$

A partir dessa desigualdade podemos obter $|\lambda| > \frac{|f'(x)|}{2}$. Como λ é constante, f' não pode se anular nem trocar de sinal em $[a, b]$, o que o isolamento garante ao produzir intervalos em que f é monótona. Newton-Raphson usa a mesma expressão, mas com $f'(x)$ no lugar de λ .

Escolha de λ . Sendo $M_1 = \max_{x \in [a, b]} |f'(x)|$, qualquer $|\lambda| > \frac{M_1}{2}$ garante $|g'(x)| < 1$. A implementação adota $|\lambda| = 1,05M_1$ por dois motivos. Primeiro, com $|\lambda| > M_1$, tem-se

$$0 < g'(x) \leq M = 1 - \frac{\min_{x \in [a, b]} |f'(x)|}{|\lambda|} < 1,$$

ou seja, g é uma contração crescente, o que simplifica a prova de que $g([a, b]) \subset [a, b]$. Segundo, a folga em relação ao mínimo $\frac{M_1}{2}$ protege contra subestimação de M_1 pela amostragem. O custo é a velocidade: M seria mínimo com $|\lambda| = \frac{M_1 + m_1}{2}$, em que $m_1 = \min |f'(x)|$.

Invariância do intervalo. Supondo f crescente e $\lambda > 0$, de $a < b$ e $f(a)f(b) < 0$ podemos derivar $f(a) < 0 < f(b)$. Com $f(a) < 0$ e $g(a) = a - \frac{f(a)}{\lambda}$, podemos derivar $g(a) > a$, e de maneira análoga, derivamos $g(b) < b$, e, como g é crescente, temos $g([a, b]) \subset [a, b]$. O caso f decrescente é análogo. Assim, o Teorema do Ponto Fixo garante convergência para a raiz única ξ a partir de qualquer $x_0 \in [a, b]$.

Cálculo e limitações. A função `pf_lambda` estima M_1 pelo maior $|f'(x)|$ em 21 pontos igualmente espaçados, usa o sinal de f' no ponto médio e multiplica por 1,05. A convergência é apenas linear, pois $g'(\xi) = 1 - \frac{f'(\xi)}{\lambda} \neq 0$. Se a raiz for múltipla, $f'(\xi) = 0$ e $g'(\xi) = 1$, e a contração se perde perto da raiz, caso discutido na Seção 4.

```
func pf_lambda(inst types.Problem) float64 {
    const samples = 20

    mid := b_candidate(inst.Start, inst.End)
    sign := math.Copysign(1, inst.Deriv(mid))
    maxAbs := math.Abs(inst.Deriv(inst.Start))
    step := (inst.End - inst.Start) / samples

    for i := 1; i <= samples; i++ {
        x := inst.Start + float64(i)*step
        d := math.Abs(inst.Deriv(x))
        if d > maxAbs {
            maxAbs = d
        }
    }

    const margem = 1.05
    return sign * maxAbs * margem
}
```

2.2.4. Newton-Raphson.

A função principal **Newton** é responsável por aplicar o algoritmo de Newton-Raphson. Esta função recebe, além dos parâmetros convencionais, uma quantidade máxima de iterações, a fim de prevenir um loop infinito em caso de não convergir. A escolha inicial da raiz é feita de forma aleatória dentro do intervalo da instância. A cada iteração uma nova raiz é calculada de acordo com a função auxiliar até que o critério de precisão seja cumprido ou o número máximo de iterações atingido.

```
func Newton(inst types.Problem, epsilon float64, kmax uint16) (float64,
uint16) {
    var a = inst.Start
    var b = inst.End
    var root float64
    var counter uint16 = 1
    for root = a + rand.Float64()*(b-a); math.Abs(inst.Function(root)) >
epsilon && counter < kmax; root = n_candidate(root, inst) {
        counter++
    }

    return root, counter
}
```

A função auxiliar **n_candidate** gera um novo candidato a raiz executando o cálculo $x - \frac{f(x)}{f'(x)}$.

```
func n_candidate(x float64, inst types.Problem) float64 {
    return x - (inst.Function(x) / inst.Deriv(x))
}
```

Ambos os métodos (Ponto Fixo e Newton-Raphson) são casos particulares da forma $\varphi(x) = x + A(x)f(x)$, diferindo apenas na escolha de $A(x)$. No Ponto Fixo, $A(x) = -\frac{1}{\lambda}$ é constante, calculado uma única vez antes das iterações; no Newton-Raphson, $A(x) = -\frac{1}{f'(x)}$ é reavaliado a cada passo, acompanhando a inclinação local de f . Essa diferença determina a ordem de convergência: no Newton, $g'(\xi) = 0$ na raiz simples, o que resulta em convergência quadrática; no Ponto Fixo, $g'(\xi) = 1 - \frac{f'(\xi)}{\lambda} \neq 0$, pois $|\lambda| > |f'(\xi)|$, de modo que a convergência é apenas linear. Em contrapartida, o Ponto Fixo tem convergência garantida a partir de qualquer ponto de $[a, b]$ sob as hipóteses do teorema e dispensa o cálculo de f' durante as iterações, enquanto o Newton só garante convergência local e exige $f'(x_k) \neq 0$ em cada passo.

2.2.5. Secante.

A função principal **Secant** é responsável por aplicar o algoritmo da Secante. Esta função recebe, além dos parâmetros convencionais, uma quantidade máxima de iterações, a fim de prevenir um loop infinito em caso de não convergir. Para este método é necessário definir dois pontos x iniciais; na implementação, x_1 é definido pela função **b_candidate** e x_2 pela função **fp_candidate**; esta escolha foi tomada para gerar ao menos 1 ponto que pode trazer maior chance de variabilidade, como é o caso do x_2 . A cada iteração do algoritmo um novo x_2 é calculado e ajustado, o x_1 movido para o antigo x_2 e novas imagens são processadas; na prática, toda iteração traça uma nova rota secante a partir de dois pontos x_1 e x_2 que atravessa a curva até x_2 se aproximar o suficiente da raiz real.

```
func Secant(inst types.Problem, epsilon float64, kmax uint16) (float64,
uint16) {
    x1 := b_candidate(inst.Start, inst.End)
    x2 := fp_candidate(inst.Start, inst.End, inst.Function(inst.Start),
inst.Function(inst.End))
    fx1 := inst.Function(x1)
    fx2 := inst.Function(x2)
    var counter uint16 = 0

    for ; math.Abs(fx2) > epsilon && math.Abs(x2-x1) > epsilon && counter <
kmax; counter++ {
        x_ := s_candidate(x1, x2, fx1, fx2)
        x1 = x2
        fx1 = fx2
        x2 = x_
        fx2 = inst.Function(x2)
    }

    return x2, counter
}
```

A função auxiliar `s_candidate` gera um novo candidato a raiz executando o cálculo

$$x_2 - \frac{f(x_2) \cdot (x_2 - x_1)}{(f(x_2) - f(x_1))}.$$

```
func s_candidate(x1, x2, fx1, fx2 float64) float64 {
    return x2 - (fx2 * (x2 - x1) / (fx2 - fx1))
}
```

2.3. Novo método: Método Caótico (Método da Corrida do Bêbado).

O novo método proposto é implementado com o uso de paralelismo, alcançado na linguagem Go por meio de *goroutines*. O modelo utiliza quatro *threads*: três responsáveis por diferentes funções de iteração e uma *thread* observadora. Esta última tem a função de incrementar o contador de iterações, que não refletirá o número real de iterações executadas pelas demais *threads*, e verificar se o valor capturado da raiz atinge a precisão desejada, retornando-o em caso positivo.

O método é denominado ‘caótico’ pelo fato de todas as *threads* compartilharem a variável com o valor atual da raiz, podendo modificá-la simultaneamente e sem restrições, o que ocasiona condições de corrida intencionais. No entanto, a *thread* observadora captura esse valor e o armazena em uma variável interna para realizar as verificações de forma segura. Isso evita o retorno de uma raiz com precisão incorreta em decorrência de uma condição de corrida do tipo *check-then-act*.

Por questão de simplicidade, optamos por inicializar `x` como a média do intervalo. O corpo da função declara a variável compartilhada, os dois acessadores usados por todas as *threads* e os dois canais de coordenação: `done`, fechado ao final para encerrar as *goroutines* de iteração, e `chResult`, pelo qual a observadora entrega o resultado.

```
func Chaotic(inst types.Problem, epsilon float64, kmax uint16) (float64,
uint16) {
    sharedRoot := (inst.Start + inst.End) / 2.0

    read := func() float64 {
```

```

    return sharedRoot
}
write := func(v float64) {
    sharedRoot = v
}

done := make(chan struct{})
chResult := make(chan types.ResultStr)

```

A primeira *goroutine* aplica a iteração de Newton. Note-se que ela não reaproveita o método `Deriv` da instância, calculando a derivada com um passo próprio de $h = 0,001$, uma ordem de grandeza menor que o $h = 10^{-2}$ empregado nos métodos sequenciais. A guarda `deriv != 0` evita a divisão por zero que interromperia a *goroutine*.

```

// goroutine de Newton
go func() {
    h := 0.001
    for {
        select {
        case <-done:
            return
        default:
            x := read()

            fx := inst.Function(x)
            fxh := inst.Function(x + h)
            deriv := (fxh - fx) / h

            if deriv != 0 {
                write(x - (fx / deriv))
            }
        }
    }
}()

```

A segunda aplica a iteração da Secante. Diferentemente das outras duas, ela precisa de memória própria: mantém em `xPrev` o ponto anterior da sua própria sequência, inicializado no extremo esquerdo do intervalo, enquanto o ponto atual é sempre lido da variável compartilhada. É justamente essa combinação entre estado privado e estado compartilhado que dá ao método seu caráter imprevisível.

```

// goroutine da Secante
go func() {
    xPrev := inst.Start
    for {
        select {
        case <-done:
            return
        default:
            xCurr := read()
            fxCurr := inst.Function(xCurr)
            fxPrev := inst.Function(xPrev)

```

```

    if fxCurr != fxPrev {
        next := (xPrev*fxCurr - xCurr*fxPrev) / (fxCurr - fxPrev)
        write(next)
        xPrev = xCurr
    }
}
}()

```

A terceira reaproveita integralmente as funções do método sequencial do Ponto Fixo, estimando λ uma única vez antes do laço com a mesma `pf_lambda` e delegando cada passo a `pf_candidate`. É a mais simples das três, pois toda a sua iteração cabe em uma linha.

```

// goroutine do Ponto Fixo
go func() {
    lambda := pf_lambda(inst)
    for {
        select {
        case <-done:
            return
        default:
            write(pf_candidate(read(), inst, lambda))
        }
    }
}()

```

Por fim, a *goroutine* observadora. Além de verificar a precisão sobre o valor capturado, ela desempenha um segundo papel de correção: se o valor compartilhado se tornar NaN, infinito ou escapar do intervalo isolado, o que ocorre quando alguma das iterações diverge, ela o descarta e reinicia a busca em um ponto aleatório de $[a, b]$, mantendo as três *threads* dentro do intervalo onde a raiz foi isolada.

```

// goroutine observadora
go func() {
    for i := uint16(1); i < kmax; i++ {
        snapshot := read()

        forDoIntervalo := snapshot < inst.Start || snapshot > inst.End
        if math.IsNaN(snapshot) || math.IsInf(snapshot, 0) ||
forDoIntervalo {
            write(inst.Start + rand.Float64()*(inst.End-inst.Start))
            continue
        }
        if math.Abs(inst.Function(snapshot)) < epsilon {
            chResult <- types.ResultStr{Root: snapshot, Iter: i}
            return
        }
    }
}()

final := <-chResult
close(done)

```

```
    return final.Root, final.Iter
}
```

3. EXPERIMENTOS

Os seis métodos foram submetidos a quatro problemas, escolhidos de modo a cobrir polinômios de graus distintos e funções transcendentais, conforme a Tabela 1. Para cada problema, o algoritmo de isolamento descrito na Seção 2 é executado primeiro, e cada método de refinamento é aplicado individualmente a cada subintervalo isolado. Nos quatro casos o isolamento retornou exatamente um subintervalo, de modo que a contagem de iterações reportada adiante corresponde a uma única busca de raiz por método.

ID	$f(x)$	$[a, b]$	h	Intervalo isolado
1	$2x^4 + 4x^3 + 3x^2 - 10x - 15$	$[0, 3]$	0.6	$[1.2, 1.8]$
2	$x^5 - 2x^4 - 9x^3 + 22x^2 + 4x - 24$	$[0, 5]$	0.7	$[1.4, 2.1]$
3	$5x^3 + x^2 - e^{1-2x} + \cos(x) + 20$	$[-5, 5]$	0.5	$[-1.0, -0.5]$
4	$x \sin(x) + 4$	$[1, 5]$	0.5	$[4.0, 4.5]$

TABELA 1. Problemas utilizados, com o passo de tabulação h e o subintervalo devolvido pelo isolamento.

A Figura 1 apresenta as quatro funções no domínio completo informado no enunciado, com os extremos dos subintervalos isolados marcados em vermelho. Como em várias delas a escala do domínio inteiro esconde a troca de sinal, a Figura 2 repete os gráficos restritos a uma vizinhança do intervalo isolado, obtida acrescentando a cada lado uma margem de 50% da largura do próprio intervalo. Os gráficos são gerados automaticamente pelo pacote **graph** a cada execução do programa.

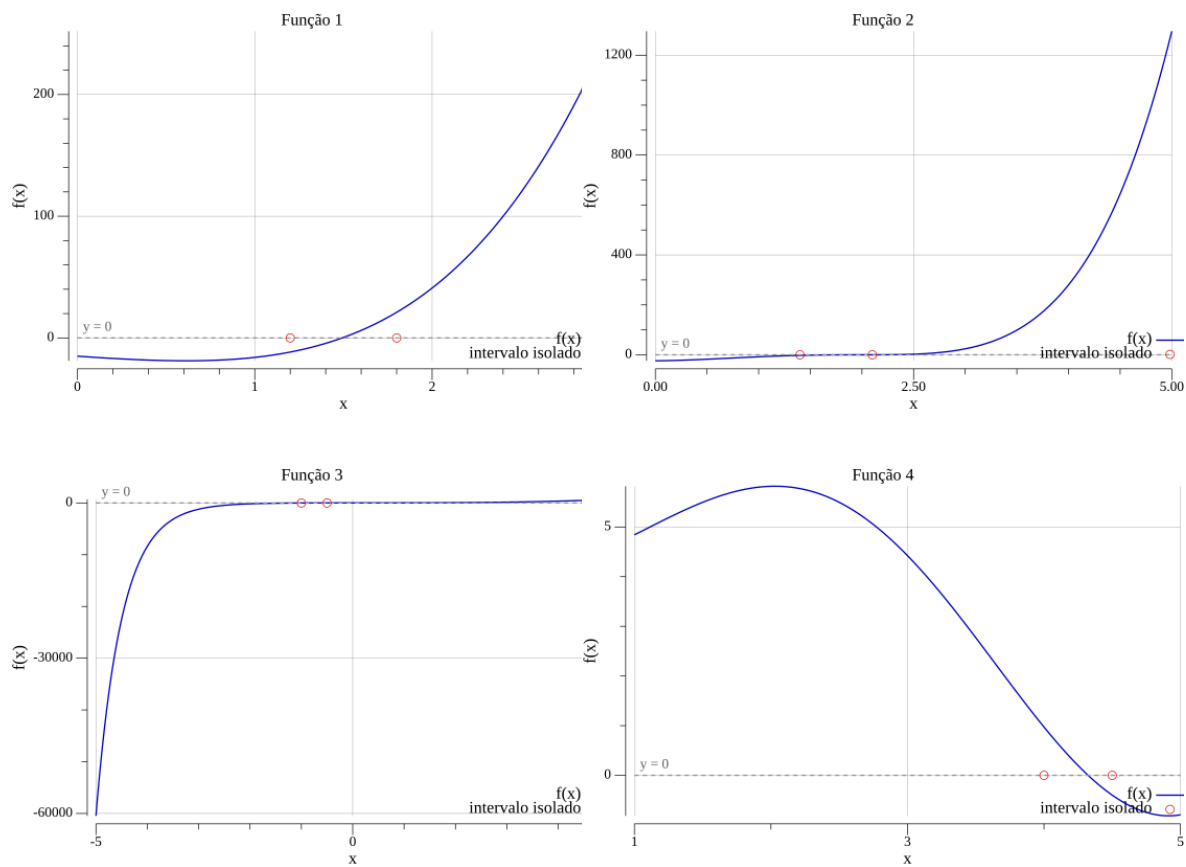


FIGURA 1. As quatro funções no domínio completo, com os extremos dos intervalos isolados em vermelho.

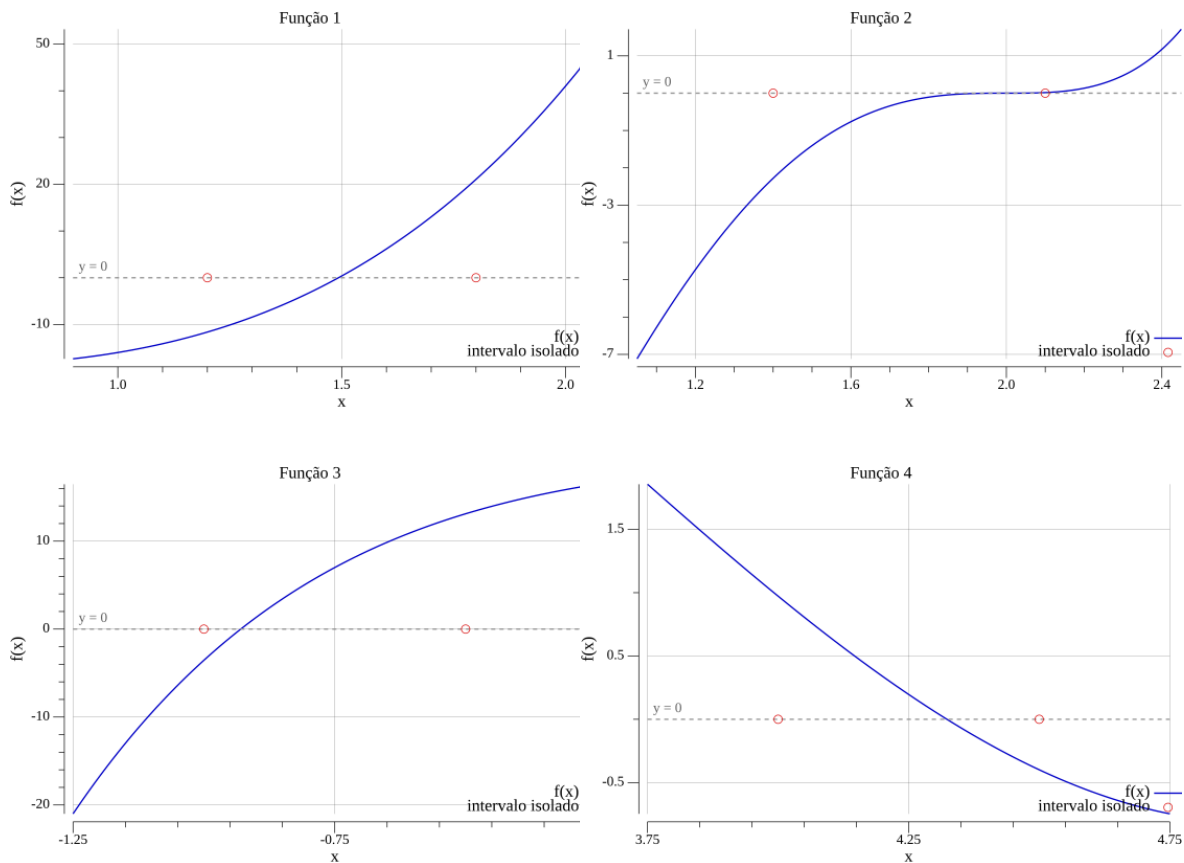


FIGURA 2. Aproximação em torno dos intervalos isolados, onde a troca de sinal se torna visível. No caso da função 2, o achatamento da curva ao redor de $x = 2$ antecipa visualmente a multiplicidade tripla da raiz, discutida na Seção 4.

Adotou-se precisão $\varepsilon = 10^{-8}$, duas ordens de grandeza mais rigorosa que o mínimo de 10^{-6} exigido pelo enunciado, e número máximo de iterações $k_{\max} = 65535$, o maior valor representável em `uint16`, tipo escolhido para o contador. Os experimentos foram executados em Go 1.27.1, linux/amd64, a partir de um binário compilado previamente com `go build`, de forma que os tempos medidos não incluem compilação.

Cada execução completa do programa foi repetida 10 vezes. A repetição se justifica porque dois dos métodos não são determinísticos: o Newton-Raphson sorteia o chute inicial dentro do intervalo isolado, e o Método Caótico depende do escalonamento das goroutines e das condições de corrida sobre a variável compartilhada. Os demais métodos partem sempre dos mesmos pontos, derivados exclusivamente dos extremos do intervalo, e portanto produzem resultados idênticos em todas as repetições, o que as 10 execuções confirmaram empiricamente, servindo como verificação de que nenhuma fonte de variabilidade involuntária foi introduzida.

Para cada método foram registradas três grandezas: o número de iterações até a satisfação do critério de parada, o tempo de parada da execução e a raiz

obtida. A partir da raiz calculou-se também o resíduo $|f(x)|$, que permite separar duas questões distintas e frequentemente confundidas, a saber quantas iterações o método consumiu e quão próximo da raiz verdadeira ele efetivamente chegou.

4. RESULTADOS

Os dados consolidados das 10 execuções aparecem nas quatro tabelas a seguir, uma por problema. Para os métodos determinísticos, os valores de iterações e de raiz foram idênticos nas 10 repetições, e o tempo médio é a única grandeza com dispersão; para o Newton-Raphson e o Método Caótico, reportam-se o mínimo, a média e o máximo observados.

Método	Iterações			Tempo méd.	Raiz	$ f(x) $
	mín	méd	máx			
Bissecção	27	27,0	27	5,69 μ s	1,492878712714	$2,1 \times 10^{-7}$
Falsa Posição	15	15,0	15	0,86 μ s	1,492878708522	$7,4 \times 10^{-9}$
Newton- -Raphson	6	6,7	7	1,05 μ s	1,492878708701	$2,0 \times 10^{-9}$
Secante	4	4,0	4	0,37 μ s	1,492878708663	$3,2 \times 10^{-11}$
Ponto Fixo	22	22,0	22	2,71 μ s	1,492878708786	$6,4 \times 10^{-9}$
Caótico	426	1568,7	8882	74,97 μ s	1,492878708663	$3,2 \times 10^{-11}$

TABELA 2. Função 1, intervalo isolado $[1.2, 1.8]$. Raiz e resíduo são médias das 10 execuções.

Método	Iterações			Tempo méd.	Raiz	$ f(x) $
	mín	méd	máx			
Bissecção	9	9,0	9	4,24 μ s	2,000195312500	$1,1 \times 10^{-10}$
Falsa Posição	34382	34382,0	34382	6918,50 μ s	2,000873444100	$1,0 \times 10^{-8}$
Newton- -Raphson	15	60,6	111	10,57 μ s	2,000000434358	$2,1 \times 10^{-14}$
Secante	17	17,0	17	1,20 μ s	2,000829924046	$8,6 \times 10^{-9}$
Ponto Fixo	65535	65535,0	65535	4913,25 μ s	1,997707656805	$1,8 \times 10^{-7}$
Caótico	516	804,1	1068	52,26 μ s	1,999298008474	$5,2 \times 10^{-9}$

TABELA 3. Função 2, intervalo isolado $[1.4, 2.1]$. O Ponto Fixo esgotou $k_{\max}$ nas 10 execuções, sem atingir a precisão exigida.

Método	Iterações			Tempo méd.	Raiz	$ f(x) $
	mín	méd	máx			
Bisseccção	27	27,0	27	5,29 μ s	-0,929560456425	$1,6 \times 10^{-7}$
Falsa Posição	9	9,0	9	0,63 μ s	-0,929560459816	$1,0 \times 10^{-9}$
Newton- -Raphson	5	6,6	8	1,17 μ s	-0,929560459803	$1,6 \times 10^{-9}$
Secante	5	5,0	5	0,48 μ s	-0,929560459838	$7,4 \times 10^{-12}$
Ponto Fixo	13	13,0	13	2,86 μ s	-0,929560459633	$9,6 \times 10^{-9}$
Caótico	290	638,7	862	47,50 μ s	-0,929560459842	$1,9 \times 10^{-10}$

 TABELA 4. Função 3, intervalo isolado $[-1.0, -0.5]$. Raiz e resíduo são médias das 10 execuções.

Método	Iterações			Tempo méd.	Raiz	$ f(x) $
	mín	méd	máx			
Bisseccção	26	26,0	26	1,42 μ s	4,323239542544	$3,0 \times 10^{-9}$
Falsa Posição	10	10,0	10	0,38 μ s	4,323239544792	$2,8 \times 10^{-9}$
Newton- -Raphson	3	5,1	6	0,33 μ s	4,323239543515	$5,1 \times 10^{-10}$
Secante	4	4,0	4	0,20 μ s	4,323239543714	$7,3 \times 10^{-13}$
Ponto Fixo	14	14,0	14	0,72 μ s	4,323239540758	$7,6 \times 10^{-9}$
Caótico	2048	3860,6	5264	56,33 μ s	4,323239543428	$7,3 \times 10^{-10}$

 TABELA 5. Função 4, intervalo isolado $[4.0, 4.5]$. Raiz e resíduo são médias das 10 execuções.

4.1. Desempenho geral dos métodos clássicos.

O método da Secante domina as demais implementações nos quatro problemas, tanto em número de iterações quanto em tempo e em resíduo, chegando a resolver as funções 1 e 4 em apenas 4 iterações. Esse desempenho decorre da combinação entre a convergência superlinear do método e a escolha dos dois chutes iniciais adotada na implementação: em vez de dois pontos arbitrários, empregam-se o ponto médio e o ponto da falsa posição do intervalo isolado, ambos já razoavelmente próximos da raiz.

A Bisseccção apresenta o comportamento previsto pela teoria, com número de iterações praticamente constante e determinado apenas pela razão entre a largura do intervalo e a precisão exigida, entre 26 e 27 iterações nas funções 1, 3 e 4. É também o único método cujo custo é insensível à forma da função, propriedade que se revela valiosa na função 2.

Um ponto merece atenção na leitura da coluna de resíduos: nas funções 1 e 3 a Bisseccção exibe os maiores valores de $|f(x)|$, da ordem de 10^{-7} , acima da precisão

nominal de 10^{-8} . Isso não indica falha, mas sim que o laço terminou pelo critério de largura do intervalo, $b - a \leq \varepsilon$, e não pelo critério de resíduo. Após 27 bisseções de um intervalo de largura 0,6, tem-se $b - a \approx 4,5 \times 10^{-9}$, de modo que o erro na abscissa é menor que 10^{-8} ; o resíduo é maior apenas porque $|f'| \approx 52$ nessa vizinhança amplifica o erro quando ele é observado na imagem. O episódio ilustra que resíduo pequeno e erro pequeno são critérios distintos, e que compará-los entre métodos exige saber qual critério interrompeu cada laço.

4.2. O caso da função 2: raiz de multiplicidade tripla.

A função 2 fatora-se exatamente como

$$f(x) = (x - 2)^3(x + 3)(x + 1),$$

o que se verifica por $f(2) = f'(2) = f''(2) = 0$ e $f'''(2) = 90$. A raiz contida no intervalo isolado $[1.4, 2.1]$ é, portanto, de multiplicidade tripla, e essa única característica explica todas as anomalias da Tabela 3.

O Ponto Fixo é o caso mais grave: esgotou as 65535 iterações nas 10 execuções e devolveu 1,997707656805, com resíduo $1,8 \times 10^{-7}$, uma ordem de grandeza acima da precisão exigida, e erro de $2,3 \times 10^{-3}$ na abscissa. A razão está na construção de λ . No intervalo isolado, f' decresce de 9,936 no extremo esquerdo até 0,4825 no extremo direito, de modo que $\lambda \approx 10,43$. Como $f'(2) = 0$, segue que $g'(2) = 1 - 0/\lambda = 1$: a constante de contração vale exatamente 1 na raiz. O método preserva a garantia de não divergir, mas perde a convergência efetiva, pois o fator de redução do erro tende a 1 conforme a iteração se aproxima da solução. É o limite teórico antecipado na descrição do método, aqui materializado.

A Falsa Posição consumiu 34382 iterações e 6,9 ms, três ordens de grandeza acima do que gastou nas outras três funções, onde bastaram de 9 a 15 iterações. O achatamento de f em torno da raiz tripla faz com que a média ponderada por $f(a)$ e $f(b)$ desloque o intervalo de forma ínfima a cada passo, mantendo um dos extremos praticamente estagnado. O critério de parada $|f(b) - f(a)| > \varepsilon$ agrava o quadro, pois essa diferença decresce lentamente justamente na região achatada.

O Newton-Raphson perde a convergência quadrática, como previsto para raízes múltiplas, passando a convergir linearmente com fator assintótico $1 - 1/3$. Isso aparece como a maior dispersão observada no método, de 15 a 111 iterações contra 3 a 8 nas demais funções. Ainda assim, foi o método mais preciso da tabela, com resíduo $2,1 \times 10^{-14}$.

A Bisseção, por sua vez, encerrou em apenas 9 iterações com resíduo $1,1 \times 10^{-10}$, mas raiz 2,000195, isto é, erro de 2×10^{-4} na abscissa. O achatamento que penaliza os demais métodos aqui produz o efeito inverso e enganoso: como $|f| \approx |x - 2|^3 \times 15$ nessa vizinhança, um erro de 2×10^{-4} em x gera resíduo da ordem de 10^{-10} , satisfazendo o critério $|f(x)| \leq \varepsilon$ muito antes de a abscissa estar de fato próxima da raiz. Trata-se de um exemplo direto do mau condicionamento de raízes múltiplas, em que o critério de resíduo deixa de ser um bom indicador de proximidade.

4.3. Comportamento do Método Caótico.

O Método Caótico converge nos quatro problemas e produz resíduos competitivos, entre 10^{-9} e 10^{-11} , equiparáveis aos da Secante nas funções 1 e 3. Notavelmente,

foi o único método que não degradou na função 2, resolvendo-a em 804 iterações médias do observador, contra as 65535 do Ponto Fixo e as 34382 da Falsa Posição. A explicação está na própria estrutura do método, pois a *goroutine* que estagna não impede as demais de escreverem na variável compartilhada, e basta que uma das três funções de iteração progrida para que o observador capture um valor aceitável.

O preço é a dispersão. O contador do observador varia de 426 a 8882 na função 1, um fator de 21 entre a melhor e a pior execução, com desvio padrão de 2580. Essa variabilidade não é um defeito da implementação, mas a manifestação direta do escalonamento não determinístico das *goroutines*. Vale reiterar que esse contador não mede o trabalho computacional realizado, já que as três *goroutines* de iteração executam livremente e em número indeterminado de vezes entre duas leituras consecutivas do observador; ele mede quantas inspeções o observador precisou fazer até encontrar um valor aceitável.

A existência das condições de corrida não é conjectural: o detector de *data races* do Go, acionado com `go run -race .`, acusa 11 corridas distintas e encerra o programa com `exit status 66`. Todas apontam para o mesmo endereço, o da variável `sharedRoot`, e para os pares de leitura e escrita dos acessadores `read` e `write` invocados pelas quatro *goroutines*. É a confirmação direta de que o comportamento descrito na Seção 2 ocorre de fato em tempo de execução, e não apenas em princípio.

Mais interessante que a confirmação é o efeito colateral da instrumentação. Sob `-race`, o observador aceita a raiz na primeira iteração nas quatro funções, contra as centenas ou milhares de inspeções necessárias na execução normal. A explicação está no custo que o detector impõe a cada acesso à memória compartilhada: como a *goroutine* observadora é a última a ser criada e cada uma de suas próprias leituras também fica mais lenta, as três *threads* de iteração acumulam um número enorme de passos antes da primeira inspeção, e o valor compartilhado já satisfaz a precisão quando ele é finalmente examinado. As raízes obtidas permanecem corretas, inclusive na função 2, onde o valor devolvido foi 2,000040930917, mais preciso que o da execução sem instrumentação. O episódio é a evidência mais clara de que o contador do observador mede latência de inspeção, e não esforço computacional.

Um resultado inesperado é a estabilidade da raiz devolvida. Embora o número de iterações varie amplamente, a raiz final foi idêntica nas 10 execuções nas funções 1, 2 e 3, e assumiu apenas dois valores distintos na função 4. Mais revelador ainda, nas funções 1 e 4 a raiz devolvida coincide dígito a dígito com a produzida pela Secante, o que sugere que o valor aceito pelo observador provém tipicamente da *goroutine* da Secante, a mais rápida das três em atingir a faixa de precisão. O caráter caótico do método afeta, portanto, o caminho e o custo, mas não o destino.

Em termos de tempo absoluto, o método é de uma a duas ordens de grandeza mais lento que os clássicos nos casos bem comportados, de 47 a 75 μ s contra frações de microssegundo, o que era esperado dado o custo de criação das quatro *goroutines* e a contenção sobre a variável compartilhada. O ganho aparece apenas no caso patológico da função 2, em que sua robustez compensa o overhead.

5. CONCLUSÃO

As implementações e experimentos realizados ao longo do trabalho foram fundamentais para compreensão teórica e prática do cálculo de raízes de funções

de qualquer ordem ou complexidade. Além dos métodos tradicionais, como Falso Positivo e Newton-Raphson, foi apresentado um método inovador baseado em programação concorrente insegura. O denominado Método Caótico, ao custo de mais iterações e tempo computacional requerido, conseguiu encontrar em todos os testes aproximações boas o suficiente para se mostrar um método valioso, competindo diretamente com métodos muito eficientes como o da Secante.

Como sugestões para trabalhos futuros, sugere-se aprimorar a etapa de isolamento de raízes a fim de evitar a situação de *loops* infinitos, instanciar a *goroutine* observadora primeiro para validar um possível ganho de desempenho ou redução de iterações e experimentar a transformação efetiva da *goroutine* observadora em uma gerenciadora, permitindo maior controle das operações entre threads, definir qual método será executado em qual instância, dentre outras características.

DEPARTAMENTO DE INFORMÁTICA E MATEMÁTICA APLICADA, UNIVERSIDADE FEDERAL DO RIO
GRANDE DO NORTE, NATAL, RN